

Japi Base Manual

INTRODUCTION

Japi Base is a small retro computer built on a Raspberry Pi Pico 2 -- the same kind of microcontroller used in many hobby projects. JAPI BASE provides the basics a program would need to turn THE PICO 2 into a computer: VGA display, keyboard input, sound output, file storage. All of that only consumes half of the Pico 2's processing capacity, leaving the other half for YOU.

PART I OF THIS MANUAL is for everyone who wants to see what Japi Base is capable of: copy the firmware to the Pico via a USB cable, add a few resistors and capacitors on a breadboard, and you can see and hear it at work.

PART II covers how to put your own files onto the machine, how to choose a keyboard layout that matches your hardware, and -- for readers who want to go further -- how to write your own program on top of the `japi_base.h` API.

The appendices at the back give a one-page hardware pin-map and a short summary of every function in `japi_base.h`, so you can quickly look something up without scanning the rest of the manual.

TABLE OF CONTENTS

PART I -- INSTALLING THE JAPI BASE DEMO

1. What Japi Base is
What's in the box, what it does, what it doesn't do.
2. What you need
The parts list and how to wire them up on a breadboard.
3. Installing the firmware on the Pico
Get a working machine in under 30 minutes.
4. A tour of the built-in demo
A walkthrough of every page in the demo cycle.
5. Troubleshooting
No picture, no sound, and other recipes.

PART II -- BEYOND THE BUILT-IN DEMO

6. Files and storage (drives A: and C:)
Putting your own files on the machine via an SD card, and what the on-chip built-in media is for.
7. Keyboard layout
QWERTY_US works out of the box; load any other layout from a .kbd file named in CONFIG.SYS on the SD card.
8. Getting set up to write your own program
Installing the Pico SDK, cloning the repo, building your first UF2.
9. Hello, Japi Base
A minimal application written from scratch.
10. VGA: text and bitmap
Characters, colours, the bitmap window, double-buffering.
11. Keyboard input
japi_get_char(), the 16-bit key codes, modifier bits.
12. Audio
The four-channel synth, raw sample writing, and how to make sound.
13. File I/O
Opening, reading, writing, and listing files from your program.
14. Tips, conventions, and pitfalls
vga_update, IRQ timing, what not to touch.
15. Where to look in the source
A quick orientation in japi_base.c and japi_base.h.

APPENDICES

- A. Quick connection lookup table
All connections at a glance plus the pins left free for you.
- B. japi_base.h API reference
Every public function, two lines each.
- C. Licence and credits
BSD-3-Clause, third-party libraries, image attributions.

PART I -- INSTALLING THE JAPI BASE DEMO

1. What Japi Base is

Japi Base is a small computer built around the Raspberry Pi Pico 2. The Pico 2 is a small microcontroller board costing about ten euros and widely used in hobby projects. Japi Base adds the things you will need to make it run like a little computer: VGA screen output, PS/2 keyboard input, sound output and file I/O. With those in place it becomes much simpler to write your own programs -- a code editor, for example -- that run on the Pico 2.

The aim is also educational. Everything is documented, the source code is small enough to read in an afternoon and nothing important is hidden behind drivers you cannot inspect. If you want to understand how a computer works at a level you can actually reason about, Japi Base lets you.

A few practical points to keep in mind so the rest of the manual makes sense:

- Japi Base runs ONE program at a time. There is no operating system, no shell, no task switcher. When you flash a UF2 file onto the Pico, that file IS what the machine does. Replace the UF2 and the machine does something else entirely. That single program can of course be a large one with several sub-programs -- a code editor, an assembler and an emulator all bundled into the same UF2, for example.
- You write programs for Japi Base on a PC and copy them to the Pico over a USB cable. There is no built-in editor or compiler on Japi Base itself; Part II of this manual covers the PC-side toolchain.
- The bundled program -- the demo that ships in the firmware -- is what you'll see the first time you install Japi Base. Part I walks through what the demo shows and how to wire up the hardware that lets you see it.

What Japi Base is NOT: it is not a high-end games console (it can do simple graphics but it is not GPU-grade) and it is not a successor or a competitor to the Raspberry Pi single-board computers (no Linux, no networking, no host-side USB). Bringing expectations from any of those will lead to disappointment; Japi Base is its own thing.

What Japi Base IS good for: a data acquisition system with integrated graphics, a BASIC interpreter, a code editor and an assembler -- in short, the kind of projects a home computer of the 1980s (Commodore 64, ZX Spectrum, ...) could do.

2. What you need

To use Japi Base you need a handful of things: the Japi Base unit itself -- building it is what the rest of this chapter is about -- a VGA monitor that can display 1024 x 768 (with its cable), a PS/2 keyboard (with its built-in cable) and a set of active (powered) speakers, headphones or a small amp. If all you want is to watch the video Japi Base produces, a VGA monitor on its own is enough; the keyboard and speakers only matter once you want to type or hear sound.

A micro-SD card is optional. Japi Base runs fine without one. The built-in demo only ever looks at a card: it shows whether one is present and lists what is on it -- it never changes the contents. Actually reading and writing files -- both on the SD card (drive A:) and on the built-in media (drive C:) inside the Pico's own flash -- is done through the Japi Base file API that your own programs use, covered in Part II (Chapter 13). Any FAT-formatted card works, of any size: prepare it on your PC and the demo will list what you put on it (see Chapter 6).

Bill of materials to build the Japi Base

These are the parts you assemble onto the breadboard to build the Japi Base unit itself:

- | | |
|--|----------|
| - 1 x Raspberry Pi Pico 2 | ~10 EUR |
| - 1 x full-size breadboard | ~3-5 EUR |
| - 1 x bidirectional logic level shifter module
(e.g. a TXS0102- or BSS138-based board, 2
channels minimum) -- protects the Pico's 3.3 V
inputs from the PS/2 keyboard's 5 V signals | ~2-3 EUR |
| - 1 x micro-SD-to-SD card adapter -- the plastic
adapter that comes with most micro-SD cards; this
is what becomes the micro-SD slot | 0 EUR |
| - 1 x 3.5 mm audio jack socket | ~1 EUR |
| - female-to-male and female-to-female jumper wires | ~3 EUR |
| - resistors and capacitors for VGA and audio
(values given with each interface below) | ~2 EUR |

Two channels on the level shifter are enough: the PS/2 keyboard uses only two signal lines (clock and data) and both are bidirectional. A four-channel module works just as well -- you simply leave the spare channels unconnected.

Wiring philosophy

This chapter builds Japi Base on a breadboard, which is the quickest way to just try it out. If you later decide to use Japi Base for a real project -- something you will keep working on for months -- it is worth rebuilding it on a piece of perfboard so nothing can shake loose, or having the dedicated Japi Base printed circuit board made. The board has been built and tested; its manufacturing files (the

gerbers, the parts list and a build guide) ship with the project, ready to send to a fab such as JLCPCB or PCBWay.

Most projects on this scale add a separate breakout board for every interface. YOU CAN GO EVEN LIGHTER : female-ended jumper wires push straight onto the pins that stick out of the VGA cable's connector and the PS/2 keyboard cable's connector, so the cables themselves become the breakouts. The other end of each jumper wire plugs into the breadboard, where the Pico, the level shifter, the audio filter and the resistor ladder for VGA all live. No soldering of the cable connectors is needed.

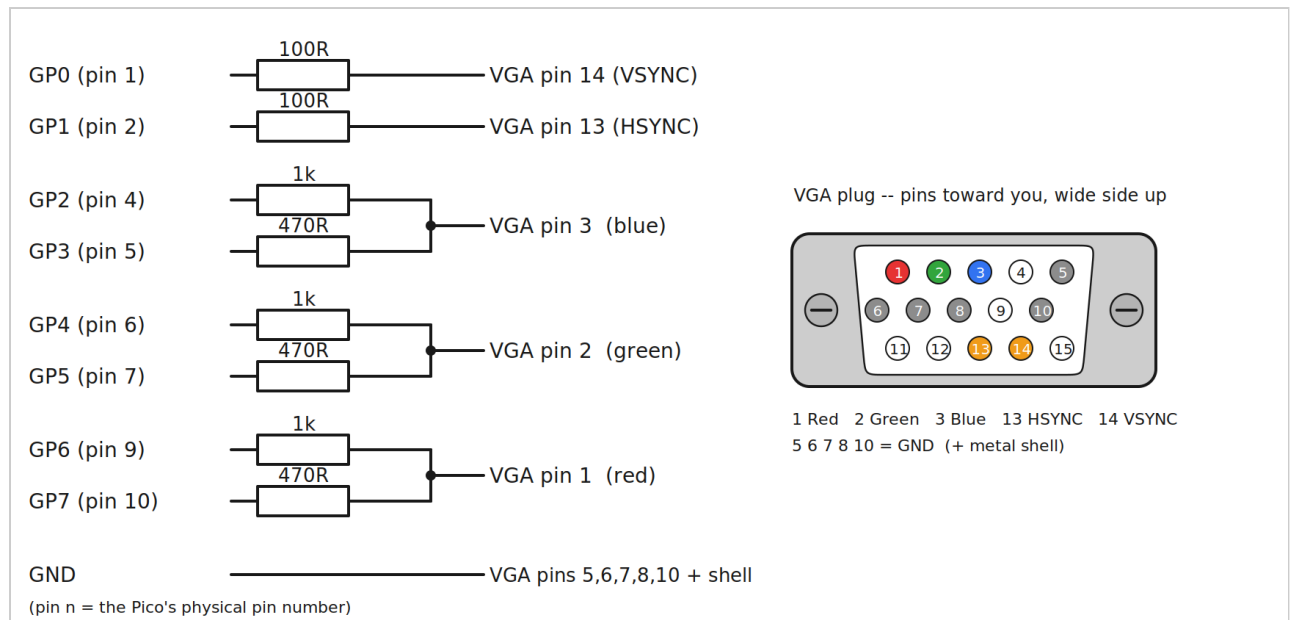
There is one place where the wiring needs extra care: the open-drain PS/2 lines sit at 5 V when idle (the keyboard pulls them up internally) and the Pico 2's GPIO inputs are absolute-maximum-rated for 3.6 V. Putting a bidirectional logic level shifter module between the keyboard and the Pico, on the clock and data lines, fixes this with one small component. Nothing else on the board needs level shifting: VGA is output-only at 3.3 V, the SD card adapter is 3.3 V native and the audio output is filtered straight into a 3.5 mm jack.

Each interface is described in full below -- the exact Pico pin for every wire and the value of every resistor and capacitor. In the diagrams "R" means ohm and "k" means kilohm (so 1k is 1000 ohm), and capacitor values are in nF (nanofarad) or uF (microfarad). Appendix A repeats every connection as a single quick-lookup table for when you just want to check one pin.

VGA

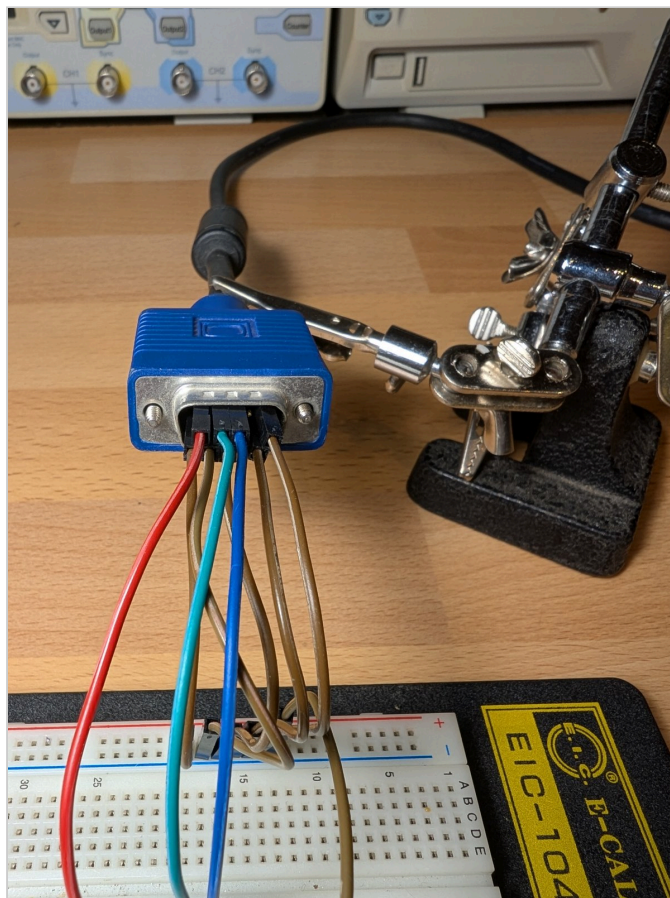
The VGA connector on the cable is a male DE-15 (sometimes called HD-15). Of its fifteen pins you only need five signals plus ground: red, green, blue, horizontal sync and vertical sync. The rest are ground returns or unused signals you can ignore.

Each colour is driven by two Pico pins joined through a small resistor ladder -- the more-significant bit through 470R, the less-significant bit through 1k -- into one VGA colour pin. Two bits per colour give four levels each, so $4 \times 4 \times 4 = 64$ colours. The two sync signals each go out through a single 100R resistor. Wire it up as shown below:



VGA resistor-DAC wiring (GP0-GP7 through their 470R/1k/100R resistors to VGA pins 1, 2, 3, 13, 14) and, beside it, the male DE-15 connector face -- pin 1 top-left, RGB on pins 1/2/3, sync on 13/14, GND on 5/6/7/8/10 plus the metal shell.

Push a female-ended jumper wire onto each of those VGA pins and run the other end to the breadboard; the resistors sit on the breadboard between the Pico pins and the VGA wires.

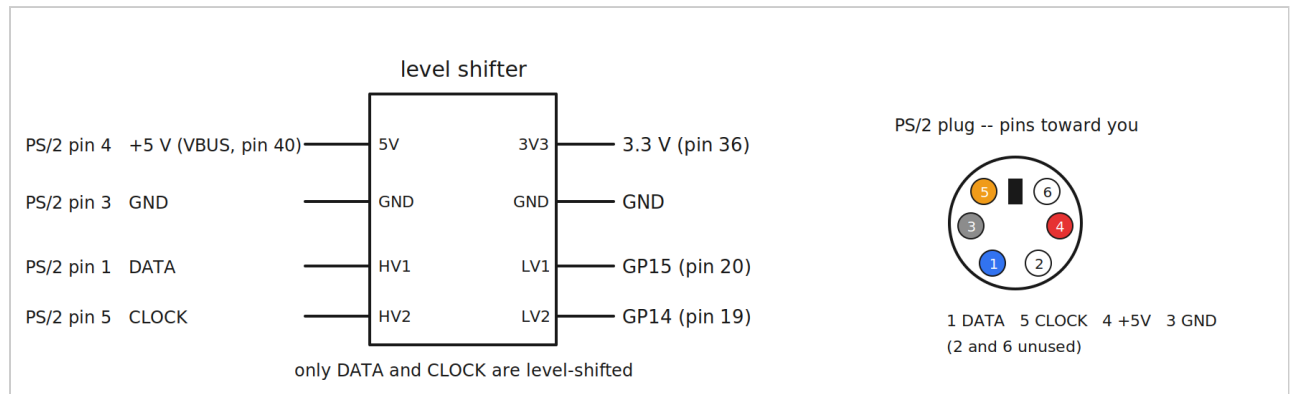


VGA cable connector with jumper wires pushed onto its pins

PS/2 keyboard

The PS/2 connector on the keyboard's own cable is a male mini-DIN-6. Four of its six pins matter -- data, clock, +5 V and ground -- and

the data and clock lines run through the level shifter on their way to the Pico. Wire it as shown below:



the level-shifter module (5V/GND/HV1/HV2 on the keyboard side, 3V3/GND/LV1/LV2 on the Pico side). DATA = PS/2 pin 1 -> HV1, LV1 -> GP15; CLOCK = PS/2 pin 5 -> HV2, LV2 -> GP14; +5 V = PS/2 pin 4 (from the Pico's VBUS); GND = PS/2 pin 3. Beside it, the mini-DIN-6 connector face.

Only the clock and data lines pass through the level shifter: their 5 V side faces the keyboard, their 3.3 V side faces the Pico's GPIO inputs. The +5 V and ground wires connect directly. Mini-DIN pin numbers are easy to mis-read, so if the keyboard stays dark, confirm the pinout against your keyboard or with a multimeter before assuming something is broken.



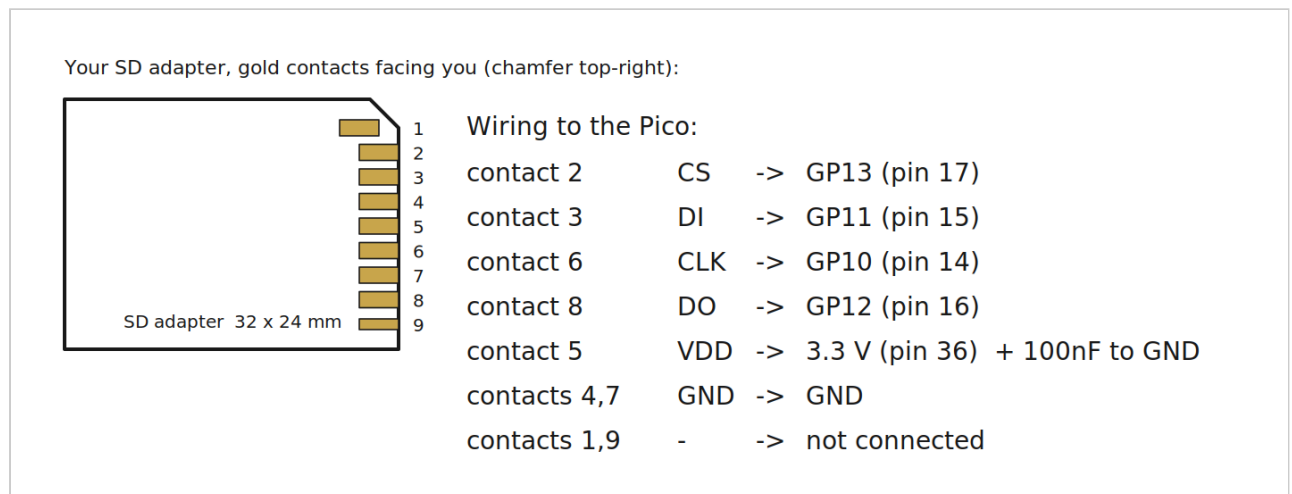
PS/2 cable connector with jumper wires and the level shifter on the breadboard

The keyboard's +5 V comes from the Pico's VBUS pin, which is simply the 5 V from the USB cable passed straight through -- it does not go through the Pico's own 3.3 V regulator. So the keyboard is really

powered by whatever you plug the USB end into, and the only limit is how much current that source can supply, shared between the Pico and the keyboard. A normal PC USB port or phone charger (500 mA or more) runs the Pico and a typical keyboard without trouble. Only a power-hungry keyboard on a weak USB source can fall short; if a keyboard refuses to wake up, give it its own 5 V supply (the Troubleshooting chapter covers this).

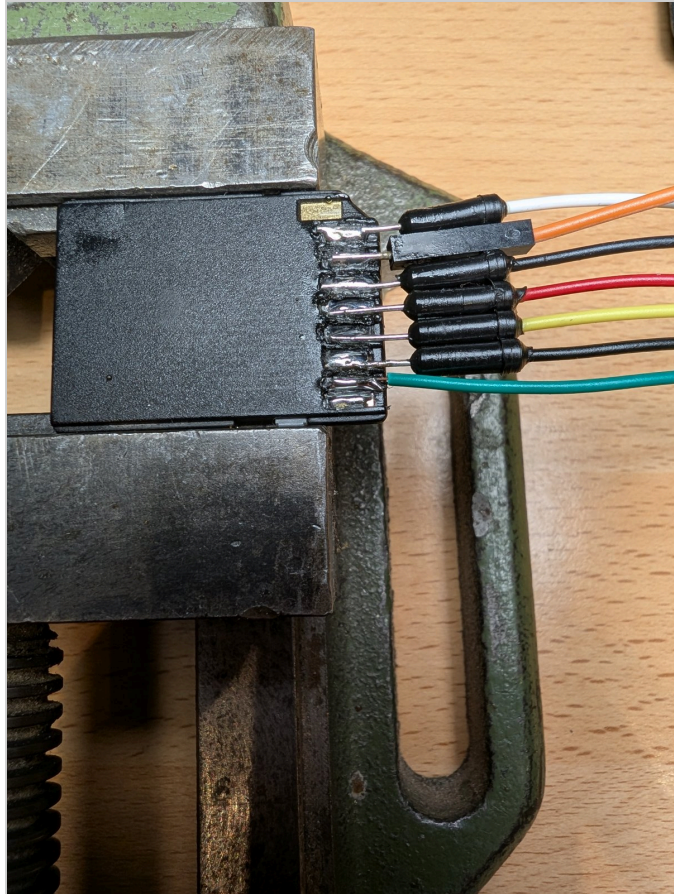
SD card

The full-size SD card adapter exposes the card's contacts as a row of flat metal pads on its back. Solder a thin wire to each pad you need (or use test-clip leads to keep the adapter solder-free) and run the wires to the breadboard; the micro-SD card then slots into the adapter as usual. Japi Base talks to it over SPI. Count the contacts from the chamfered corner (contact 1) and wire them as shown below:



the SD adapter drawn to scale, contacts numbered 1-9 from the chamfer, with the wiring -- contact 2 = CS -> GP13, 3 = DI -> GP11, 6 = CLK -> GP10, 8 = DO -> GP12, 5 = VDD -> 3.3 V (with a 100nF capacitor to GND), 4 and 7 = GND, 1 and 9 unused.

The 100nF capacitor sits right next to the adapter, between 3.3 V and ground, to steady the card's supply. No level shifter is needed: SD cards are 3.3 V native. The Pico pins are also available as the PIN_SD_SCK / PIN_SD_MOSI / PIN_SD_MISO / PIN_SD_SS names at the top of japi_base.h, so you can change the wiring later if you want.

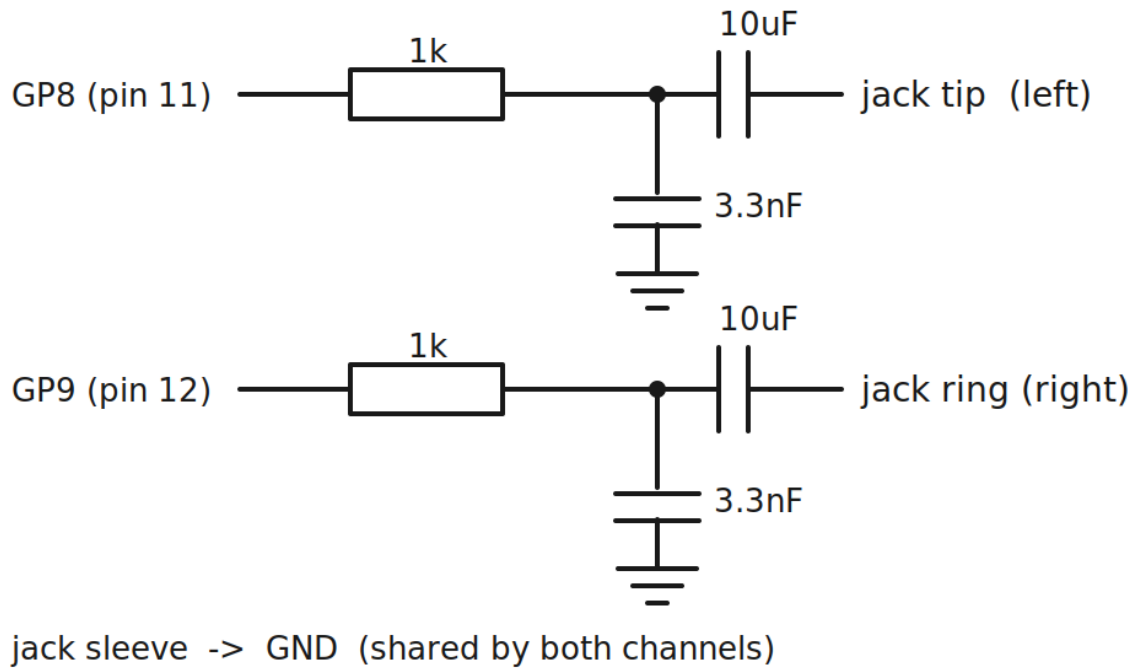


SD card adapter wired into the breadboard

If the soldering or clipping turns out too fiddly, an Adafruit "Micro SD SPI/SDIO Card Breakout Board -- 3V ONLY!" is a one-piece alternative: a small PCB with the slot, a 3.3 V regulator and header pins ready to wire with ordinary jumpers.

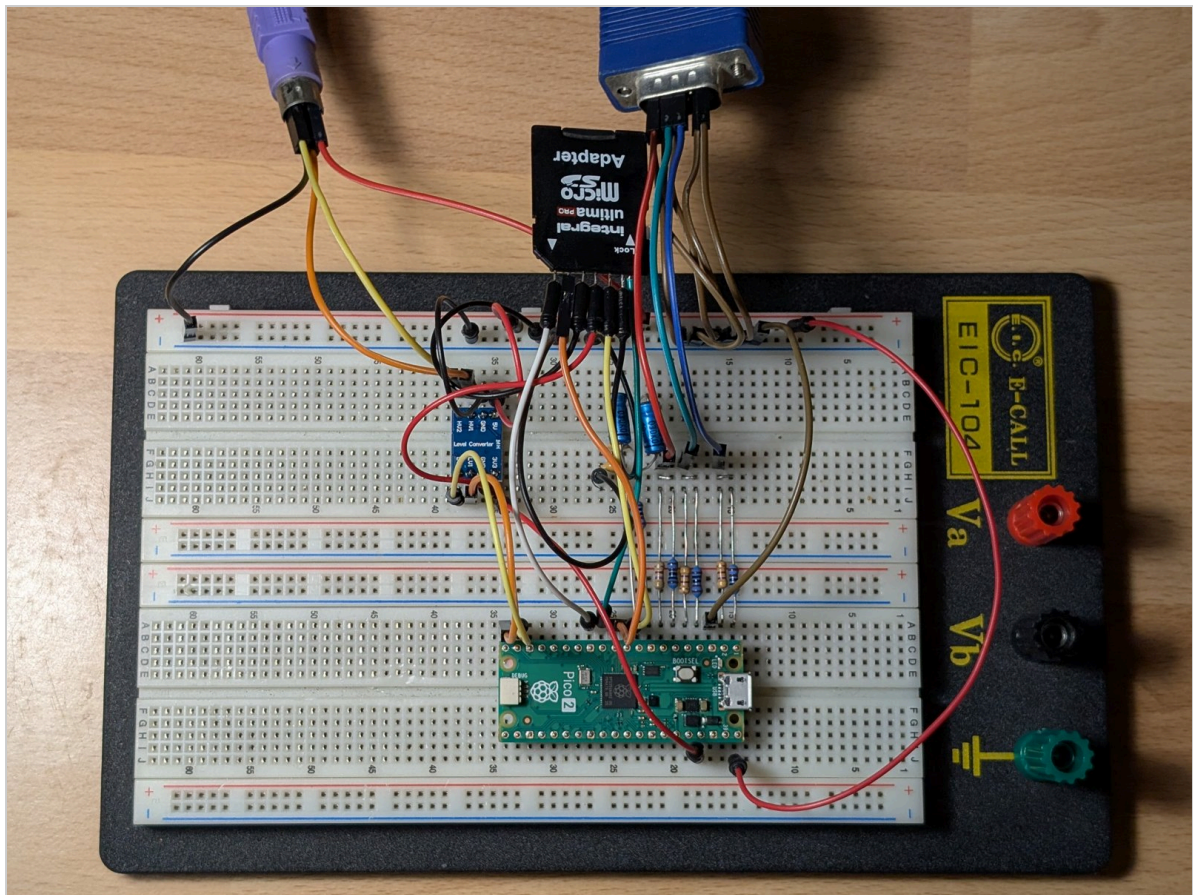
Audio

The Pico 2 has no analog audio output of its own; Japi Base makes stereo sound with two PWM pins, one per channel, each smoothed by a small RC filter before it reaches the 3.5 mm jack:



In each channel the 1k resistor and the 3.3nF capacitor form a low-pass filter (about 48 kHz) that smooths the PWM; the 10uF capacitor in series blocks DC. The filter is gentle on purpose -- your speakers, headphones or amp do the real work -- but do not skip it, or the output is harsh square-wave edges. It is meant for active (amplified) speakers or headphones.

When all four interfaces are wired, the level shifter is in place and the resistors and capacitors are seated, your breadboard should look roughly like this:



overview of the full known-good build on a double full-length breadboard

Power the Pico through its USB connector and you are ready for Chapter 3 -- flashing the firmware.

3. Installing the firmware on the Pico

This is the quickest part. The firmware is a single file -- `japi_base.uf2` -- and copying it onto the Pico takes under a minute. You need no special software, just a file manager.

1. Download `japi_base.uf2` from the Releases page of the Japi Base project on GitHub (the newest release is at the top).
2. Hold down the small BOOTSEL button on the Pico 2 and, while keeping it pressed, plug the Pico into your computer with a USB cable. Let go of the button once it is plugged in.
3. The Pico now shows up on your computer as a USB drive called RP2350, exactly like a memory stick. Drag `japi_base.uf2` onto that drive (or copy and paste it there).
4. The instant the file finishes copying, the Pico restarts by itself and begins running Japi Base. The RP2350 drive vanishes -- that is normal; it means the firmware was accepted.

If your breadboard is wired up and a monitor is connected, the demo's showcase page should appear within a second or two. If the screen stays black, or you get a picture that rolls or has no colour, turn

to the Troubleshooting chapter.

Installing a different program later works in exactly the same way: hold B00TSEL, plug in, drag the new .uf2 onto the RP2350 drive. The new file simply replaces whatever was there before, so you can switch between programs as often as you like.

4. A tour of the built-in demo

The firmware ships with a demo that shows off what Japi Base can do. It has five pages and you move to the next one by pressing any key, so you can linger on each as long as you want. After the last page it wraps back to the first.

Showcase

The opening page is a sampler. It shows the full 64-colour palette, the complete character set (letters, digits, punctuation and the box-drawing characters used to frame windows), a few overlapping text windows, a bar chart and a small drawing area in which the four sound waveforms are plotted. While you look at it the Tetris theme plays in four-channel stereo, so this page also confirms that your sound wiring works.

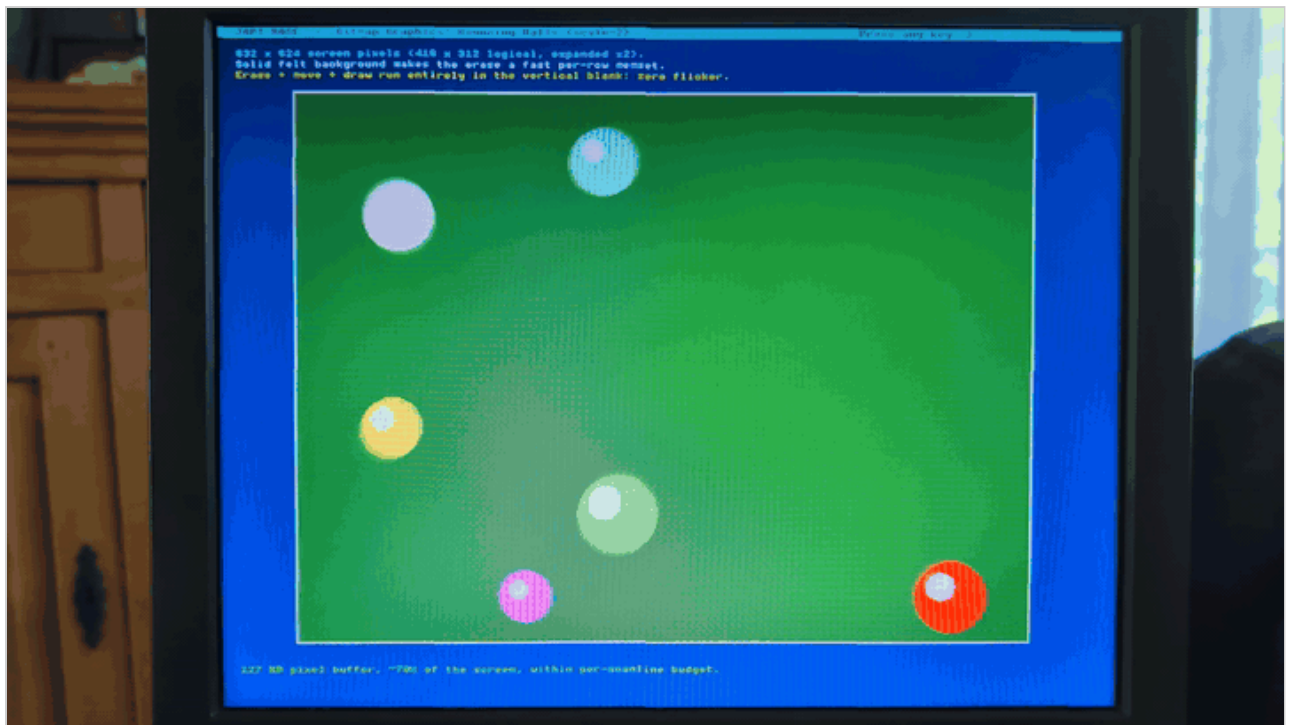


the showcase page on a real monitor

Bouncing balls

A handful of balls bounce around on a solid background. There is

nothing to do here -- it is included to show that smooth, flicker-free animation at 60 frames per second is possible on the machine.



bouncing balls (an animation in the online version)

The Starry Night

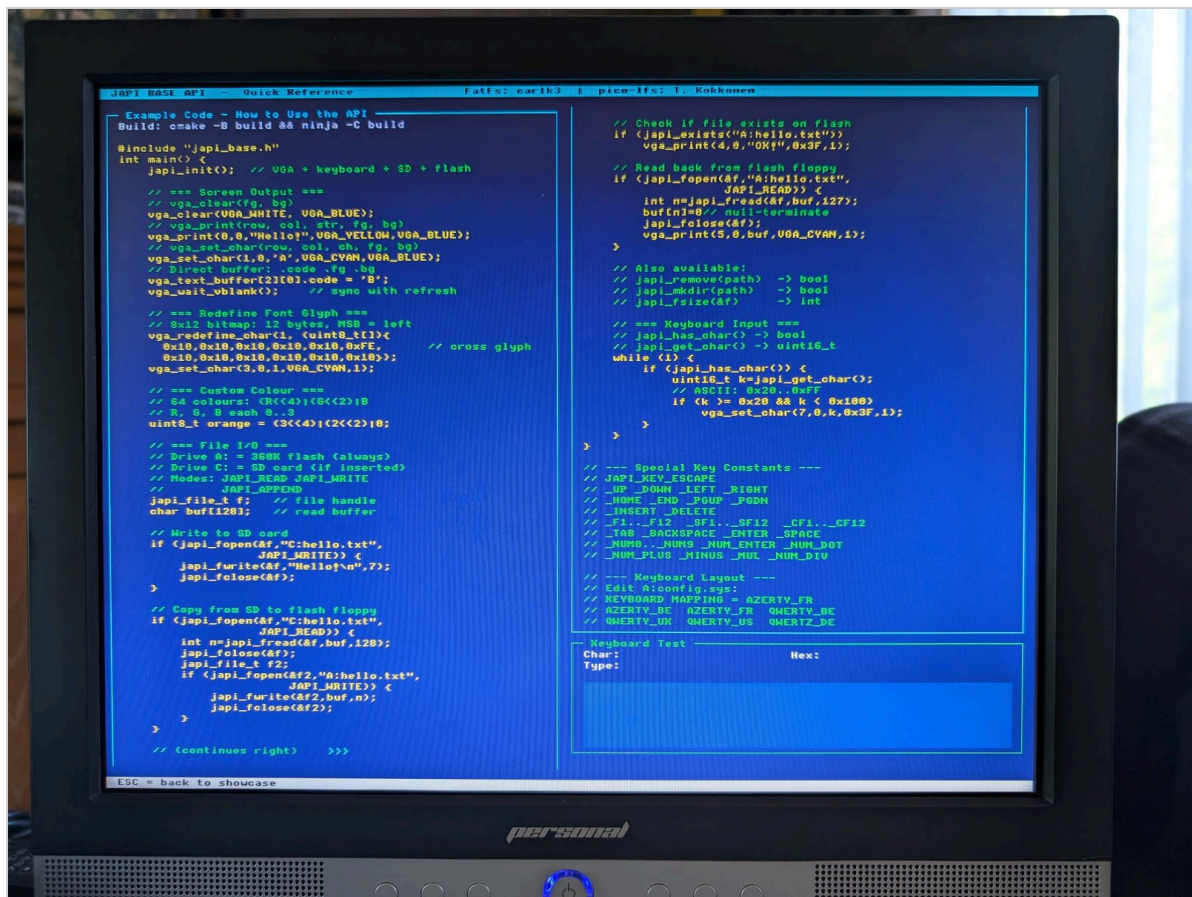
Van Gogh's painting, shown almost full screen. The original has far more colours than Japi Base can display, so it has been converted into the 64-colour palette with a technique called dithering, which mixes nearby colours to fool the eye into seeing shades that are not really there. The picture is baked into the firmware; nothing is read from a card.



The Starry Night, dithered

API quick reference

A one-screen cheat sheet of the most useful things a program can ask Japi Base to do. It doubles as a live keyboard test: press any key and the page shows you exactly what that key produced, which is handy for checking that your keyboard layout is the one you expect (see the Keyboard layout chapter). If a micro-SD card is in the slot, this page also lists the first few files it finds on it.



the API reference and live key test

CPU benchmark

The Pico runs a fixed amount of calculation and times how long it takes. Press Shift+T to step the processor speed up and t to step it down, through 260, 324 and 390 MHz. The throughput figure rises and falls with the speed while the computed result stays exactly the same -- proof that the chip keeps calculating correctly even at the highest speed. The picture itself never changes, because the video timing is held constant whatever the processor speed.

5. Troubleshooting

Most first-time problems come down to a single wire in the wrong hole. The entries below are grouped by what you see (or do not see), each with the most likely cause and what to try. Work through them in order; the earlier ones rule out the most common mistakes.

No picture at all (black screen)

Likely cause: the firmware is not running, the monitor is on the wrong input, or the sync and ground wires are off.

Try: confirm the firmware took -- the RP2350 drive should have vanished after you copied the .uf2. Set the monitor to its VGA input. Check the two sync wires (GP0 through 100R to VGA pin 14, GP1 through 100R to pin 13) and that the Pico's GND reaches the VGA ground pins and the connector's metal shell.

Picture rolls, flickers or the monitor says "out of range"

Likely cause: the monitor cannot lock onto the 1024 x 768 @ 60 Hz

signal.

Try: a different monitor that lists 1024 x 768 @ 60 Hz among its supported modes -- most do. If one particular monitor still refuses, there is a one-column "padding" workaround described in Part II.

Wrong colours, or a colour missing

Likely cause: a swapped or loose resistor on one of the colour pairs.

Try: for each colour, check that the MSB pin goes through 470R and the LSB pin through 1k into the same VGA pin (pin 1 red, pin 2 green, pin 3 blue). A whole colour missing usually means that pair never reaches its VGA pin.

No sound, or only one channel

Likely cause: the audio filter, the jack wiring, or unpowered speakers.

Try: make sure your speakers are switched on and have their own power. Check that each channel has its 1k resistor, its 3.3nF capacitor to ground and its 10uF capacitor to the jack (tip = left, ring = right, sleeve = ground). One silent channel points at one missing part on that side.

The keyboard does nothing

Likely cause: the level shifter, its power, or a swapped clock and data line.

Try: check that the level shifter has 5 V on its HV side and 3.3 V on its LV side, that data goes to GP15 and clock to GP14 and that the keyboard's +5 V comes from the Pico's VBUS pin. An older, power-hungry keyboard on a weak USB supply may need its own 5 V source.

The keyboard types the wrong characters

Likely cause: the selected layout does not match your keyboard.

Try: the live key test on the demo's API reference page shows what each key produces. If it is wrong, choose the matching layout with a config.sys file (see the Keyboard layout chapter).

The SD card is not detected, or no files show up

Likely cause: the card is not FAT-formatted, is not seated properly, or a data wire is off.

Try: format the card as FAT on your PC and copy a file or two onto it. Reseat it in the adapter. Check the four SPI wires against the SD figure in Chapter 2. Japi Base boots fine without a card, so an empty slot is not itself a fault.

PART II -- BEYOND THE BUILT-IN DEMO

6. Files and storage (drives A: and C:)

Japi Base can store files and it borrows the old DOS idea of drive letters to keep things simple. There are two drives -- A: and C: -- and they behave quite differently.

Drive A: -- the SD card

A: is whatever micro-SD card you have pushed into the slot. It is an ordinary card in the FAT or exFAT format that cameras and phones use, so your PC can read and write it freely. Anything you copy onto the card from your PC shows up on A: the next time Japi Base reads it. You can insert or swap the card while the machine is running -- it is picked up on the next file operation, with no reboot needed. Because you can pop it out and carry it to your PC, A: is the drive for larger files and for moving things between your PC and Japi Base.

Drive C: -- the built-in media

C: is a small storage area, about 360 KB, that lives inside the Pico's own flash memory. It is always there, even with no SD card. Whatever a program writes to it stays put when you switch the power off. Think of it as a small built-in store that is always present.

There is one limitation: you can only write to C: from a program running on Japi Base. You cannot drag files onto it from your PC, because it is not a removable medium -- it is sealed inside the Pico. So C: is the place for a program to keep its own small files (settings, a high-score table, a short note) that should survive a reboot.

Putting files on A: from your PC

1. Take a micro-SD card of any common size. Japi Base reads both the FAT and exFAT formats that cards already come in, so you normally need not reformat it at all.
2. Copy a file or two onto it.
3. Push the card into the adapter on your Japi Base and power up.

To check that Japi Base sees the files, open the demo's API reference page (page four of the demo). It shows a small "Files on A:\\" panel listing the first few files on the card. If the panel reads "(no SD card inserted)", the card is not making contact -- reseal it and check the SD wiring. If it reads "(SD card is empty)", the card is seen but has no files on it yet.

Screenshots -- the PrintScreen key

Press PrintScreen at any moment and Japi Base writes a picture of whatever is on the monitor to the SD card. It works everywhere -- in the demo, in your own program, even while graphics are being drawn -- because the platform itself takes the picture, not the

running program. The file is an ordinary BMP image named screenshot0001.bmp, screenshot0002.bmp and so on, so any PC opens it with a double-click.

The number is a running counter kept on C:, the built-in media. Because it lives in the sealed flash and not on the card, it keeps climbing across reboots and across card swaps, and a number is never reused -- just like an old film camera that simply numbered every photo in order. Throw a screenshot away and its number stays gone; the next shot takes the next number.

A screenshot is about 768 KB, so the screen freezes for a moment while it is written to the card -- that brief pause is normal. An SD card must be present, since that is where the picture is saved.

Reading and writing files from your own programs uses these same two drives through one shared set of functions; that is covered in the File I/O chapter later in this manual.

7. Keyboard layout

Most keyboards in the world are US QWERTY, and that is the layout Japi Base uses out of the box. If you have such a keyboard you can skip this chapter entirely -- everything already matches. If your keyboard is a Belgian or French AZERTY, a German QWERTZ, or another national layout, this chapter shows how to make the machine agree with the symbols printed on your keys. It takes two small files on an SD card and about a minute.

What a "layout" is

A PS/2 keyboard does not send letters. Each key sends a small number, the same number whatever country the keyboard was made for. Turning that number into an "a", a "5" or a "@" is the job of a layout: a lookup table that says which key means which character. A US, a Belgian and a German keyboard send the same numbers for the physical keys but carry different symbols on their key caps, so each one needs its own table to read correctly.

In practice the letters A to Z and the digits usually land in the same place on every layout. It is mostly the punctuation -- the brackets, the slash, the @ sign, the accented letters -- that moves around. So a wrong layout still types; it just puts the wrong symbol on some keys.

The built-in layout

Japi Base has exactly one layout built in: QWERTY_US, the US QWERTY keyboard. It lives inside the firmware, so it is always there and needs no files at all. Nothing else is built in. Every other layout is supplied as a small file that you put on a drive, which keeps the machine empty and tidy for the majority who never need to change anything.

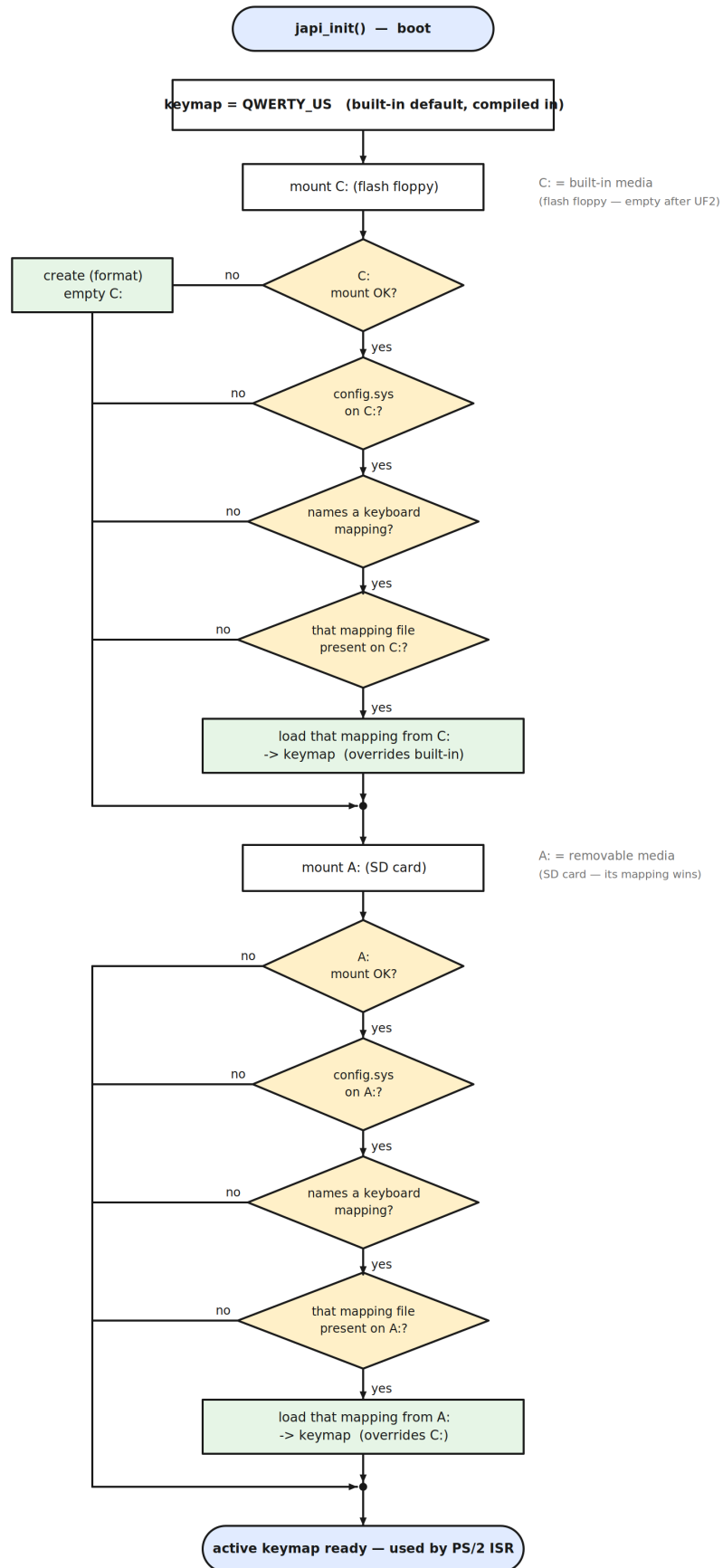
Choosing another layout

To use a different layout you provide two files:

CONFIG.SYS	a one-line text file that names the layout you want
NAME.kbd	the layout table itself (NAME matches the name above)

At start-up Japi Base looks for these files first on the built-in media (drive C:) then on the removable SD card (drive A:). If both name a layout the SD card wins. That makes the SD card the easy, swappable way to carry your layout from one machine to another; the built-in media is for when you would rather bake the choice into one particular machine (covered further down).

Japi Base — keyboard layout selection at boot



how Japi Base chooses the keyboard layout at start-up -- the built-in QWERTY_US first, then the built-in media (drive C:), then the removable SD card (drive A:) which wins

The CONFIG.SYS file holds a single line, exactly like this:

```
KEYBOARD MAPPING = AZERTY_BE
```

A few things matter here. The name after the "=" must match the name of the .kbd file (so AZERTY_BE goes with AZERTY_BE.kbd). Capitals count: type the name exactly as shown. Keep the spaces around the "=" sign. One line is enough.

The layouts that come with Japi Base

Six ready-made layouts ship with Japi Base. Each is a file you copy onto your card:

AZERTY_BE.kbd	Belgian AZERTY
AZERTY_FR.kbd	French AZERTY
QWERTY_BE.kbd	Belgian QWERTY
QWERTY_UK.kbd	British QWERTY
QWERTY_US.kbd	US QWERTY (the same one that is built in)
QWERTZ_DE.kbd	German QWERTZ

You will find these files together with Japi Base, in its keyboard layouts folder. Copy the one you need; you can ignore the rest.

Making an SD card for your keyboard (the easy way)

Here is the whole recipe, using a Belgian AZERTY keyboard as the example. For any other layout, swap the name AZERTY_BE for the one you want.

1. Take any micro-SD card that Japi Base can read (the FAT or exFAT cards from Chapter 6 -- almost certainly the card you already have).
2. Copy AZERTY_BE.kbd to the top level (the "root") of the card, not inside a folder.
3. In the same place, make a plain text file named CONFIG.SYS holding the single line:

```
KEYBOARD MAPPING = AZERTY_BE
```

4. Put the card into Japi Base and switch it on.

To check it worked, open the demo's API reference page (page four of the demo). It has a small live keyboard test: press a few of the keys whose symbols differ between US and your layout -- the ones with brackets, the slash, the accented letters -- and confirm the right characters appear. If they do, you are done; the card can stay in.

Keeping the layout without a card

If you would rather not leave an SD card in the slot, you can put the same two files on the built-in media (drive C:) instead. From then on the machine uses your layout on its own, with no card needed. Writing to drive C: has to be done from a program running on Japi Base -- you cannot drag files onto it from a PC, as Chapter 6 explained -- so this

step belongs to Part II; the File I/O chapter shows how. Until you get there, the SD card is the simplest route.

When the layout does not change

If the characters are still wrong after a reboot, run down this list:

- The name in CONFIG.SYS and the .kbd file name must match exactly, capitals included.
- Both files must sit in the root of the card, not in a folder.
- The line must read `KEYBOARD MAPPING = NAME` with the spaces and nothing else on it.
- The .kbd file for that name must actually be on the card.
- Remember the SD card wins over the built-in media. A layout left on the built-in media cannot override one named on the card.

Rolling your own layout

A .kbd file is simply a 768-byte table that lists the character for every key. Building one by hand for a keyboard layout we do not offer yet in the standard set is a small programming job rather than a copy-and-paste one; it is touched on later in the manual. If you do make a working layout for a keyboard we do not cover yet, send it to us and we will add it to the standard set in our repository so other people can use it too. Almost everyone, though, is served by one of the six layouts above.

8. Getting set up to write your own program

Part I needed nothing but a finished UF2 file. From here on you build that file yourself, so this chapter sets up the small toolchain on your PC that turns C source into a Pico 2 program. It is a one-time job. To prove the toolchain works we build the demo you already know from its own source, before you change a single line. When you finish this chapter you will have produced your own `japi_base.uf2` -- the same file Part I had you flash -- and you will be ready to write a program of your own in the next chapter.

Everything here happens on your PC, not on Japi Base. You write and build on the computer, then copy the result to the Pico over USB, just as you installed the firmware in Chapter 3.

What the toolchain is

Five pieces do the work:

- a C compiler that produces code for the Pico (`arm-none-eabi-gcc`)
- CMake, which reads the project's build description
- Ninja, which runs the actual compile and link steps
- Git, to download the source
- the Raspberry Pi Pico SDK, the support library from Raspberry Pi that the firmware is built on top of

You install the first four once. The SDK you download once and point an environment variable at, so every Pico project on your machine can

find it.

Step 1 -- install the tools

On Linux (Debian, Ubuntu and relatives) one line installs the lot:

```
sudo apt install cmake ninja-build gcc-arm-none-eabi \
git build-essential
```

On macOS, with Homebrew (from brew.sh):

```
brew install cmake ninja arm-none-eabi-gcc git
```

On Windows the simplest route is Raspberry Pi's official "Pico install" package, which bundles the compiler, CMake, Ninja and the SDK in one installer; search for "Raspberry Pi Pico Windows installer". The rest of this chapter then works the same from its terminal.

Step 2 -- download the Pico SDK

The SDK is the support library every Pico program uses. Download it once with Git, including its sub-parts:

```
git clone --recurse-submodules \
https://github.com/raspberrypi/pico-sdk.git
```

Japi Base needs SDK version 2 or newer, because the Pico 2 (the RP2350 chip) did not exist before then. A fresh clone gives you that.

Now tell every build where the SDK lives by setting the PICO_SDK_PATH environment variable to that folder. For the current terminal:

```
export PICO_SDK_PATH=~/.pico-sdk          (Linux / macOS)
```

To make it stick, add that same line to your shell's start-up file (~/.bashrc or ~/.zshrc). On Windows the official installer sets this for you.

Step 3 -- get the Japi Base source

Clone the repository:

```
git clone https://github.com/JanFromBelgium/japi-base.git
```

Everything for the machine itself lives in the folder "Japi Base Pico 2" inside it. The third-party libraries are already included, so there is nothing else to fetch.

Step 4 -- build the demo from source

Go into the firmware folder, make a build directory and let CMake and Ninja do the rest:

```
cd "Japi Base Pico 2"
mkdir build
cd build
```

```
cmake -G Ninja ..
ninja
```

The cmake step is the slow one -- it works out how everything fits together -- and ninja then compiles and links. When it finishes you have build/japi_base.uf2, the very file Part I told you to flash.

Step 5 -- flash your own build

This is the same flashing step as Chapter 3, now with the UF2 you just built. Hold the BOOTSEL button on the Pico 2 while you plug it into USB; it appears as a small USB drive. Copy the file onto it:

```
cp japi_base.uf2 /media/<you>/RP2350/      (Linux example)
```

or drag japi_base.uf2 onto the drive in your file manager. The Pico reboots and runs the demo -- the same demo as before, but now proven to come from a toolchain you control.

If you see the familiar demo, your setup is correct and you are ready to write your own program. If a step failed, the usual cause is that PICO_SDK_PATH does not point at the SDK folder, or the SDK is older than version 2; check those first, then run cmake and ninja again.

The next chapter writes the smallest possible Japi Base program from an empty file, so you can see every part that makes one work.

9. Hello, Japi Base

With the toolchain from Chapter 8 in place, this chapter writes the smallest complete Japi Base program: it puts a couple of lines of text on the screen, waits for a key, then says goodbye. It is short enough to print in full, so here it is. After the listing we build it, run it, then walk through it line by line -- by the end you will understand every part of a working program, including the one idea that surprises most newcomers: drawing and showing are two separate steps.

The program

Create a file called hello.c in the "Japi Base Pico 2" folder, next to demo.c, and put this in it:

```
#include "japi_base.h"

int main(void) {
    japi_init();

    vga_clear(VGA_WHITE, VGA_DARK_BLUE);
    vga_print(2, 4, "Hello, Japi Base!", VGA_YELLOW, VGA_DARK_BLUE);
    vga_print(4, 4, "Press any key to finish.", VGA_WHITE, VGA_DARK_BLUE);
    vga_update();                          // show what we just drew

    while (!japi_has_char()) { }           // wait for a key
    japi_get_char();                       // take it out of the queue
```

```

    vga_print(6, 4, "Bye! Press the Pico's reset button to run it again.",
              VGA_CYAN, VGA_DARK_BLUE);
    vga_update();                               // show the goodbye line

    while (1) { }                               // nothing left to do
}

```

Building and running it

The firmware is built from the source files listed in CMakeLists.txt, and the file that holds main() is demo.c. To run your program instead of the demo, tell the build to use hello.c in its place:

1. Open CMakeLists.txt in the "Japi Base Pico 2" folder.
2. Find the line that reads demo.c in the list of source files and change it to hello.c (leave japi_base.c and third_party_libs.c as they are).
3. In your build folder, run ninja again. It notices the changed CMakeLists.txt and reconfigures on its own:

```
ninja
```

The result is still called build/japi_base.uf2 -- that is just the name of the program, not the demo. Flash it exactly as in Chapter 8 (hold BOOTSEL, copy the UF2 across). Your two lines of text appear; press any key and the goodbye line shows. To get the demo back, change that one line in CMakeLists.txt from hello.c to demo.c and run ninja once more.

Walking through it

```
#include "japi_base.h"
```

This makes every Japi Base name available to your program -- the functions such as vga_print and the colour names such as VGA_YELLOW. Every Japi Base program begins with this line.

```
int main(void)
```

main is where a C program starts. Japi Base calls it once. Until it returns (and ours never does) main is your whole program.

```
japi_init();
```

The single call that starts the machine: it sets the CPU clock, brings up the VGA picture, the keyboard and the SD card, and starts the engine on the Pico's second processor core. That engine paints the screen and reads the keyboard for you in the background. Nothing works before japi_init, so it is always the first thing main does.

vga_clear and the two vga_print calls

These draw, but here is the surprising part: they do not put anything on the screen yet. Japi Base keeps an off-screen copy of the whole screen for you, and these functions only change that copy. The monitor is being painted from a different copy, so for the moment it shows nothing new.

`vga_clear` fills the off-screen copy with blanks on a background colour. The screen is a grid of character cells, 127 columns across by 64 rows down, and every cell has two colours -- a foreground (the character) and a background -- which is why `vga_clear` takes two. The two `vga_print` calls then write text: the first two numbers are the position (row 2, column 4, counting from row 0 at the top and column 0 at the left), then the text, then its foreground and background colours. The colour names come from `japi_base.h`; Chapter 10 lists them and shows how to mix your own.

```
vga_update();
```

This is the line that actually shows your drawing, and it answers the question the previous lines raise -- how does any of it reach the screen? The monitor redraws the whole picture sixty times a second. Between two of those passes there is a brief pause, the "vertical blank", when nothing is being drawn. `vga_update()` waits for that pause and then copies your off-screen version onto the copy the monitor reads. Because the copy lands entirely within the pause, the screen changes all at once -- you never catch it half-updated, so there is no flicker or tearing.

So the pattern is always the same: change the screen as much as you like with `vga_clear`, `vga_print` and `vga_set_char`, then call `vga_update()` once to show all of it together. Nothing you draw is visible until you do. The drawing functions themselves never wait or display anything; `vga_update` is the one call that makes your work appear.

```
while (!japi_has_char()) { }
```

Now we wait for the user. `japi_has_char()` is true the moment a key is waiting to be read. The loop body is empty -- it just spins until a key arrives. That works because the keyboard is filled by the engine on the other core, not by this loop, so this core is free to wait. We do not call `vga_update()` here because the screen is not changing: once a picture has been shown, the engine keeps displaying it on its own, with no further help from us.

```
japi_get_char();
```

`japi_has_char()` only tells you that a key is waiting; `japi_get_char()` actually takes it and hands back its code. For an ordinary key that is the character itself, like 'a' or '5'; for a special key it is a named value such as `JAPI_KEY_ESCAPE` or `JAPI_KEY_UP`. Here we throw the key away -- we only wanted to know that one was pressed -- so we ignore the value it returns. Chapter 11 puts it to use.

the goodbye line, then `vga_update()` again

Same two steps once more: a `vga_print` draws the farewell into the off-screen copy, and `vga_update()` shows it. Draw, then show.

```
while (1) { }
```

Japi Base runs one program with no operating system behind it to hand control back to. If `main` returned, the machine would simply stop. So

when a program has nothing left to do it loops forever -- while (1) means "keep going" with an empty body. The goodbye screen stays up because the engine keeps displaying the last picture we showed. Pressing the Pico's reset button starts the program again from the top.

That is a whole program. The next chapter stays on the screen and goes further with text and colour, then opens the bitmap window for pixel graphics.

10. VGA: text and bitmap

Chapter 9 put a few words on the screen. This chapter covers the screen properly: how the text grid and its colours work, how to invent your own characters, and -- when letters are not enough -- how to open a window of real pixels for drawing. One rule from Chapter 9 holds throughout: you change an off-screen copy with the functions below, then call `vga_update()` once to show the result.

The character grid

The screen is a grid of character cells, 127 columns across (0 to 126) by 64 rows down (0 to 63). Each cell holds three things: a character code that says which shape to draw, a foreground colour for that shape and a background colour behind it.

You met two ways to fill cells already:

```
vga_set_char(row, col, code, fg, bg);    // one cell
vga_print(row, col, "text", fg, bg);    // a string, left to right
```

`vga_set_char` places a single character by its code -- a number from 0 to 255 into the built-in font. Codes 32 to 126 are the ordinary letters, digits and punctuation; the rest of the font holds box-drawing lines, arrows and block shapes (the demo's frames are built from them). `vga_print` is a convenience: it takes a normal C string and writes it one character per cell, stopping at the right edge -- text does not wrap to the next line.

`vga_clear(fg, bg)` fills every cell with a blank on the given background, which is the usual way to start a fresh screen.

Colour: the 64-colour palette

A colour is six bits: two for red, two for green and two for blue. Two bits give four levels per channel (0, 1, 2, 3), so the whole palette is $4 \times 4 \times 4 = 64$ colours. Both the foreground and the background of every cell can be any of them.

`japi_base.h` predefines the common ones:

```
VGA_BLACK   VGA_WHITE   VGA_RED      VGA_GREEN   VGA_BLUE
VGA_YELLOW  VGA_CYAN    VGA_MAGENTA VGA_DARK_BLUE
```


For any other shade you build the value yourself from its three levels with this pattern:

```
uint8_t colour = (red << 4) | (green << 2) | blue;    // each 0..3
```

For example a warm orange is full red, medium green, no blue:

```
uint8_t orange = (3 << 4) | (2 << 2) | 0;
vga_print(5, 4, "sunset", orange, VGA_BLACK);
```

Inventing your own characters

The font has 256 character shapes, each eight pixels wide by twelve tall. You can replace any of them at run time:

```
vga_redefine_char(code, bitmap);
```

bitmap is an array of twelve bytes, one per row from top to bottom. Within a byte the highest bit (0x80) is the leftmost pixel and the lowest bit (0x01) is the eighth pixel; a 1 lights the pixel in the foreground colour, a 0 leaves the background. For instance, a small right-pointing arrow:

```
uint8_t arrow[12] = {
    0x00, 0x08, 0x0C, 0xFE, 0xFF, 0xFE,
    0x0C, 0x08, 0x00, 0x00, 0x00, 0x00,
};
vga_redefine_char(1, arrow);           // shape 1 is now an arrow
vga_print(10, 4, "\x01 this way", VGA_YELLOW, VGA_DARK_BLUE);
vga_update();
```

Two things to keep in mind. The change affects every cell that uses that code, anywhere on the screen, because they all read the same font -- so pick a code you are not otherwise using (a high, unused one is safest). And the change lasts until you redefine the shape again or reset the machine.

The bitmap window: real pixels

Characters cannot light a single pixel wherever you like. For that you open a bitmap window: a rectangular block of the screen that becomes a free drawing canvas of individual pixels.

```
bool ok = japi_bitmap_open(col, row, w_chars, h_chars, scale,
                           double_buffered);
```

col and row are the window's top-left corner in character cells; w_chars and h_chars are its size, also in cells, so the window still lines up with the text grid around it. scale chooses how big a logical pixel is:

- scale 1: one logical pixel is one screen pixel. The canvas is w_chars*8 wide by h_chars*12 tall.
- scale 2: one logical pixel becomes a 2x2 block on screen, so the

canvas covers the same cells with half the width and half the height in logical pixels -- and a quarter of the memory. At scale 2 the width in cells must be even.

The last argument, `double_buffered`, chooses how the canvas reaches the screen. With `false` there is a single buffer: each pixel you set shows on the next `vga_update()`, which uses the least memory and suits a still image or a data plot, though a fast animation can tear. With `true` you get two buffers -- you draw into a hidden back buffer that swaps to the screen whole on `vga_update()`, so animation is tear-free, at the cost of twice the memory. Pick `false` for pictures, `true` for games.

Memory is the only real limit. A bitmap is one byte per logical pixel (two buffers' worth when double-buffered), taken from the same heap as your program's variables -- there is no fixed cap. A full-screen single buffer is about 760 KB, so a large canvas usually wants scale 2, or a smaller window, or single buffering. `japi_bitmap_open` returns `false` if the buffer will not fit or the scale-2 even-width rule is broken, so it is always worth checking the result.

Once open, you draw with:

```
japi_bitmap_clear(colour);          // fill the whole canvas
japi_bitmap_pixel(x, y, colour);    // set one pixel
```

`x` runs from 0 at the left, `y` from 0 at the top, and `colour` is one of the same 64 values as the text. To find the canvas size in logical pixels, ask `japi_bitmap_width()` and `japi_bitmap_height()`. When you need speed, `japi_bitmap_buffer()` hands you the raw bytes to write straight into (one per pixel, `width*height` of them).

A small example -- a yellow diagonal on a black square:

```
if (japi_bitmap_open(40, 20, 30, 20, 1, false)) { // 30x20 cells
    japi_bitmap_clear(VGA_BLACK);
    int n = japi_bitmap_width();
    if (japi_bitmap_height() < n) n = japi_bitmap_height();
    for (int i = 0; i < n; i++)
        japi_bitmap_pixel(i, i, VGA_YELLOW);
    vga_update();                                // show it, as always
}
```

When you are done with the window, `japi_bitmap_close()` releases it and the cells return to ordinary text.

That is the whole screen: cells with colours, your own shapes when you want them and a pixel canvas when you need one. The next chapter leaves the screen for a moment and looks properly at reading the keyboard.

11. Keyboard input

Chapter 9 used the keyboard in the simplest possible way: wait for a key, then carry on. This chapter reads it for real -- ordinary typing, the arrow and function keys, plus the Ctrl, Shift and Alt combinations -- so your program can respond to whatever the user presses.

The basic pattern

Two functions do the work, and you have met both:

```
japi_has_char();    // is a key waiting?  (true / false)
japi_get_char();    // take the next key and return its 16-bit code
```

Keystrokes pile up in a small queue as they are typed, so you will not miss one while your program is busy elsewhere. `japi_has_char()` tells you whether the queue holds anything; `japi_get_char()` removes the oldest key and hands back its code. The usual shape is: when a key is waiting, take it and decide what it means.

An example: move a marker

This program puts a star on the screen and moves it with the arrow keys until you press Esc:

```
#include "japi_base.h"

int main(void) {
    japi_init();

    int r = 10, c = 20;
    vga_clear(VGA_BLACK, VGA_BLACK);
    vga_set_char(r, c, '*', VGA_YELLOW, VGA_BLACK);
    vga_update();

    for (;;) {
        while (!japi_has_char()) { }           // wait for a key
        uint16_t k = japi_get_char();
        if (k == JAPI_KEY_ESCAPE) break;

        vga_set_char(r, c, ' ', VGA_BLACK, VGA_BLACK); // erase
        if (k == JAPI_KEY_UP    && r > 0)    r--;
        if (k == JAPI_KEY_DOWN  && r < 63)    r++;
        if (k == JAPI_KEY_LEFT  && c > 0)    c--;
        if (k == JAPI_KEY_RIGHT && c < 126) c++;
        vga_set_char(r, c, '*', VGA_YELLOW, VGA_BLACK); // redraw
        vga_update();
    }

    while (1) { }
}
```

Build and run it the same way as Chapter 9 (your own `.c`, one line in `CMakeLists.txt`, `ninja`, `flash`). The star jumps one cell per arrow press and the screen updates each time, because every change ends with `vga_update()`.

What the 16-bit codes mean

`japi_get_char()` always returns a 16-bit number, and there are two kinds.

Ordinary characters keep their everyday value, below 256. A letter comes back just as you typed it ('a' or 'A', depending on Shift), a digit as '5', punctuation as itself. The four everyday control keys have names so you need not remember their numbers: `JAPI_KEY_ENTER`, `JAPI_KEY_TAB`, `JAPI_KEY_BACKSPACE` and `JAPI_KEY_SPACE`. So to react to plain typing you compare against ordinary character literals:

```
if (k == 'q')          ...    // the q key
if (k == JAPI_KEY_ENTER) ...    // Return
```

Special keys -- the arrows, Home and End, Page Up and Page Down, Insert and Delete, Escape, the function keys F1 to F12, the numeric keypad -- have no everyday value, so each gets a name of its own at 256 or above: `JAPI_KEY_UP`, `JAPI_KEY_HOME`, `JAPI_KEY_F1`, `JAPI_KEY_ESCAPE`, `JAPI_KEY_NUM0` and so on. You used four of them in the example. The full list lives in `japi_base.h` and in Appendix B.

Because the special codes sit at 256 and up while ordinary characters stay below, one test tells them apart:

```
if (k < 256) { /* a normal character */ }
else          { /* a named special key */ }
```

Modifier combinations

Holding Ctrl, Alt or Shift while pressing another key folds into the same code, so you test for the whole combination at once:

- Ctrl with a key: `JAPI_KEY_CTRL('S')` is Ctrl+S, `JAPI_KEY_CTRL('Q')` is Ctrl+Q. The natural fit for application shortcuts.
- Alt with a letter: `JAPI_KEY_ALT('F')` is Alt+F, the usual way to open a "File" menu.
- Shift or Ctrl with a navigation key gives the selection variants an editor needs: `JAPI_KEY_SUP` (Shift+Up), `JAPI_KEY_CUP` (Ctrl+Up) and so on.

A few habits are kept on purpose: Ctrl+H, Ctrl+I and Ctrl+M still arrive as Backspace, Tab and Enter, because that is what those combinations have always meant on a terminal.

Live modifier state

Sometimes you do not want to wait for a key at all -- you just want to know, right now, whether Shift or Ctrl is being held, for instance to light an indicator in an editor's status bar. `japi_kbd_modifier_state()` returns a byte with one bit per modifier:

```
uint8_t m = japi_kbd_modifier_state();
if (m & JAPI_MOD_CTRL_L) ...    // left Ctrl is down right now
```

The bit names (JAPI_MOD_SHIFT_L, JAPI_MOD_CTRL_L, JAPI_MOD_ALTGR and the rest) are listed in `japi_base.h`.

Is a key held right now? (games)

`japi_has_char` and `japi_get_char` read keys one at a time from a queue, which is perfect for typing but awkward for a game where the player holds several keys at once -- thrust and turn together, say. For that, ask whether a particular key is down at this instant:

```
if (japi_keydown('a'))          ... // the A key is held
if (japi_keydown(JAPI_KEY_LEFT)) ... // left arrow is held
if (japi_keydown(JAPI_KEY_MOD_SHIFT)) ... // either Shift is held
```

`japi_keydown` takes the same code `japi_get_char` would return for that key and reports true while it is physically down. Two things to note. First, it is shift-independent, so you pass the base character ('a', never 'A'). Second, it reads the live hardware state rather than the queue -- so several `japi_keydown` calls can be true at once and a key you never took with `japi_get_char` still registers. It is the natural fit for movement and for action keys you want held down.

That covers reading the keyboard. The next chapter turns to sound: the four-channel synth and how to make notes and effects.

12. Audio

Japi Base has a small four-channel synthesizer that plays sound in the background. It ranges from a one-line beep to four shaped voices at once, in stereo. Best of all, it is non-blocking: you start a sound and your program carries straight on while the synth keeps it going on its own.

A beep in one line

The quickest way to make sound is a single call:

```
japi_play(NOTE_C5, 200);    // play middle-high C for 200 ms
```

`japi_play` takes a note and a length in milliseconds, starts it, then stops it for you when the time is up. It returns immediately -- the 200 ms is how long you will hear the note, not how long the call takes.

Notes have names, from `NOTE_C2` (low) up to `NOTE_B7` (high), with the sharps in between (`NOTE_CS4` is C-sharp in the fourth octave). `NOTE_C4` is middle C. There is also `NOTE_REST`, which is silence -- handy when you step through a melody and want a gap.

Four channels and a chord

The synth has four channels, numbered 0 to 3, and they all sound at the same time. `japi_play` always uses channel 0; to choose a channel yourself use `japi_play_ch`. Put a different note on three channels and you have a chord:

```
japi_play_ch(0, NOTE_C4, 800); // C
```

```
japi_play_ch(1, NOTE_E4, 800); // E -- together: a C major chord
japi_play_ch(2, NOTE_G4, 800); // G
```

Shaping a voice

Each channel is a little instrument you can set up before you sound it.
Five calls shape the voice:

```
japi_sound_wave(ch, type); // JAPI_WAVE_SINE, _SQUARE, _SAW, _TRIANGLE
japi_sound_volume(ch, vol); // 0 (silent) .. 255 (loudest)
japi_sound_pan(ch, pan); // 0 = full left, 128 = centre, 255 = right
japi_sound_freq(ch, hz); // set pitch in hertz, not a note
japi_sound_envelope(ch, attack_ms, decay_ms, sustain, release_ms);
```

The envelope is the shape of the note over time: it swells in over attack milliseconds, eases down over decay to a held level (sustain, 0 to 255), and after you let go it fades away over release. A short attack with a high sustain is an organ; a sharp attack with a low sustain and a long release is more like a plucked string.

Having set the voice, you start and stop the note by hand:

```
japi_sound_note_on(ch); // begin -- the note holds until...
japi_sound_note_off(ch); // ...you release it (it then fades)
```

Putting it together, a sawtooth note held and then released:

```
japi_sound_wave(0, JAPI_WAVE_SAW);
japi_sound_volume(0, 200);
japi_sound_pan(0, 128);
japi_sound_envelope(0, 10, 120, 160, 300); // the A-D-S-R shape
japi_sound_freq(0, 220); // 220 Hz = the note A3
japi_sound_note_on(0);
/* ... your program does other things; the note plays on ... */
japi_sound_note_off(0);
```

You rarely need all of this from cold: every channel starts as a sine wave at volume 200, centred, with a gentle envelope, which is exactly why `japi_play` makes a pleasant sound with no setup at all.

Silence

To cut all four channels at once -- the panic button at the end of a program or between tunes -- call:

```
japi_sound_off();
```

Streaming your own sound (raw PCM)

The four voices cover notes and effects, but sometimes you have actual sample data to play -- a recorded sound, a wave you compute yourself, a tune decoded from a file. For that, feed raw PCM straight to the output:

```
int japi_audio_stream(const int16_t *buf, int nframes); // mono
int japi_audio_stream_stereo(const int16_t *buf, int n); // L,R pairs
```

```
int  japi_audio_stream_space(void);    // free frames in the buffer
void japi_audio_stream_stop(void);    // give the synth back
```

Samples are signed 16-bit at about 48 kHz (the screen's line rate). The mono call plays each frame on both speakers; the stereo call takes interleaved left,right pairs. While you are streaming, the synth steps aside and the output is yours; japi_audio_stream_stop hands it back.

The two stream calls return how many frames they actually accepted, which can be fewer than you offered when the buffer is nearly full -- so keep the rest and push it next time. japi_audio_stream_space reports how much room is free, so a program can top the buffer up at its own pace without ever blocking:

```
while (frames_left > 0) {
    int room = japi_audio_stream_space();
    if (room > frames_left) room = frames_left;
    int done = japi_audio_stream(cursor, room);
    cursor += done; frames_left -= done;
    vga_update();                // get on with the frame
}
```

(On the PC simulator the stream calls are accepted but silent; sound is a Pico-only feature.)

That is sound. The final pair of chapters return to your own programs: the next one reads and writes files, then we close with tips and a tour of the source.

13. File I/O

Chapter 6 looked at the two drives from the outside -- A: the removable SD card, C: the built-in media. This chapter reads and writes them from inside your own program. One small set of functions does it, and the same calls work on either drive: you only change the letter at the front of the name.

Paths and drives

A path is the drive letter, a colon, then the file name:

```
"A:save.dat"      a file on the SD card
"C:scores.dat"    a file on the built-in media
```

Folders, if you make them, are separated with a forward slash, as in "A:games/snake.sav". Remember from Chapter 6 that you can only write to C: from a program like this one -- which is exactly what makes it the right place for a program's own saved data: the kind that should stay with the machine even when the SD card is swapped or absent.

Opening, reading, writing, closing

You work through a file handle, a variable of type japi_file_t. Open it, use it, then close it:


```
japi_file_t f;
if (japi_fopen(&f, "C:scores.dat", JAPI_WRITE)) {
    ... read or write ...
    japi_fclose(&f);
}
```

japi_fopen returns true if the file was opened. The mode says what you intend:

```
JAPI_READ      open an existing file to read it (fails if it is not
                there)
JAPI_WRITE     create the file, or replace it if it already exists
JAPI_APPEND    create the file, or add to the end of an existing one
JAPI_READ | JAPI_WRITE  open for reading and writing at once,
                        without throwing away what is there (the file is created
                        if absent). This is the mode for random access -- see
                        "Random access" below.
```

Then you move bytes:

```
int n = japi_fread(&f, buffer, size);    // returns bytes actually read
int m = japi_fwrite(&f, buffer, size);    // returns bytes written
```

Both work in raw bytes, so they handle plain text and binary data alike. japi_fread returns how many bytes it actually got, which is zero once you reach the end of the file. Always japi_fclose when you are done; on a file you wrote, closing is also what makes sure everything reaches the drive.

An example: a high-score table

Five scores live in an array. We save the whole array to the built-in media in one write, then load it straight back into another array:

```
#include "japi_base.h"
#include <stdio.h>

int main(void) {
    japi_init();
    vga_clear(VGA_WHITE, VGA_DARK_BLUE);

    int scores[5] = { 1200, 900, 750, 400, 100 };

    japi_file_t f;
    if (japi_fopen(&f, "C:scores.dat", JAPI_WRITE)) {
        japi_fwrite(&f, scores, sizeof(scores));    // the whole array
        japi_fclose(&f);
    }

    int loaded[5] = { 0 };
    if (japi_fopen(&f, "C:scores.dat", JAPI_READ)) {
        japi_fread(&f, loaded, sizeof(loaded));
    }
}
```

```

        japi_fclose(&f);
    }

    for (int i = 0; i < 5; i++) {
        char line[32];
        sprintf(line, "%d. %d points", i + 1, loaded[i]);
        vga_print(2 + i, 4, line, VGA_YELLOW, VGA_DARK_BLUE);
    }
    vga_update();
    while (1) { }
}

```

Because the data is a fixed block of five integers, one `japi_fwrite` saves it and one `japi_fread` reads it back -- no parsing needed. (`sprintf`, from the standard C library, is just turning each number into text so `vga_print` can show it.) Run it once and the file is created; power-cycle the Pico and run it again -- the scores load straight back, because the built-in media keeps them.

Random access: updating a file in place

The reads and writes so far move forward from the start of the file. To jump to any point instead -- to update one record in a data file without rewriting the whole thing -- open the file for both reading and writing and move the position with `japi_fseek`:

```

japi_file_t f;
if (japi_fopen(&f, "C:save.dat", JAPI_READ | JAPI_WRITE)) {
    japi_fseek(&f, record_no * sizeof(record)); // jump to it
    japi_fread(&f, &record, sizeof(record));    // read the record
    ... change it ...
    japi_fseek(&f, record_no * sizeof(record)); // jump back
    japi_fwrite(&f, &record, sizeof(record));   // write it back
    japi_fclose(&f);
}

```

`japi_fseek` takes a byte offset from the start of the file (0 is the first byte). With `JAPI_READ | JAPI_WRITE` the file is created if it is not there and -- the point of this mode -- existing data is kept, so you can read a record, change it and write it straight back. A record is just a fixed-size block, so its offset is `record_no * record_size`, which is all you need for a simple database, a save file with slots or a log you index into.

A few more calls

A handful of small helpers round out the set:

```

japi_exists("C:scores.dat") // true if the file is there
japi_remove("C:scores.dat") // delete a file, true on success
japi_mkdir("A:games")       // make a folder
japi_rmdir("A:games")       // remove an empty folder
japi_rename("A:old.txt", "A:new.txt") // rename or move on one drive
japi_fsize(&f)              // size in bytes of an open file

```

japi_rename also moves a file to another folder on the same drive -- it cannot cross between A: and C: (copy then delete for that) -- and it handles whole directories too. japi_rmdir removes only a folder that is already empty.

Listing a drive

When you do not know the file names in advance, walk the directory. Open it, read entries one at a time until there are no more, then close it:

```
japi_dir_t d;
if (japi_opendir(&d, "A:")) {           // the SD card
    char name[64];
    int row = 4;
    while (japi_readdir(&d, name, sizeof(name)))
        vga_print(row++, 4, name, VGA_WHITE, VGA_DARK_BLUE);
    japi_closedir(&d);
}
```

japi_readdir fills name with the next entry and returns true; it returns false when the directory is exhausted. The "." and ".." entries are not reported, so you get only real files and folders.

Two things to keep in mind: drive A: only works when a card is inserted (japi_fopen simply returns false otherwise, so a check is enough), and nothing here translates text -- a newline is whatever byte you write. The next chapter collects the conventions and the few sharp edges worth knowing before you build something larger.

14. Tips, conventions, and pitfalls

The functions in the earlier chapters are most of what you need. This chapter gathers the few things that are not obvious from the function list: the habits that keep a program tidy and the handful of boundaries that keep it out of the engine's way.

Always show your drawing

It bears repeating, because it is the most common first mistake: nothing you draw with vga_clear, vga_print, vga_set_char or the bitmap calls appears until you call vga_update(). Draw as much as you like, then call it once. In an animation you call it once per finished frame; on a still screen you call it once and leave it.

Core 0 is yours, Core 1 is the engine

Your program runs on Core 0, the first of the Pico 2's two processor cores. The second core, Core 1, runs the Japi Base engine -- the video, the keyboard and the sound -- and it is not yours to use. Write all of your own code for Core 0; do not launch work of your own on Core 1.

Do not switch interrupts off for long

The engine does its work through an interrupt that fires on every

scanline of the display. Ordinary code never notices, but if you turn interrupts off and keep them off for more than the briefest moment that scanline interrupt cannot run, and the picture, the sound or the keyboard will stutter. You almost never need to disable interrupts at all; if you do, keep the critical section tiny.

Leave the engine's hardware alone

A few pieces of the Pico are claimed by the engine and must be left untouched: the system clock, the PIO0 block and the two DMA interrupts it uses (one for the SD card, one for the video). Plenty is still free for you, though: Core 0, both spare PIO blocks (PIO1 and PIO2) with their four state machines each and the DMA channels -- as long as you drive those by polling rather than through their interrupts.

Change the CPU speed the right way

Japi Base can run at three speeds: 260 MHz (a safe floor), 324 MHz (the default) and 390 MHz (an opt-in boost). Choose one with a single call:

```
japi_set_cpu_clock(390);    // saves the choice and reboots into it
```

Never set the clock registers yourself -- that is one of the reserved pieces above. Switching is safe even on a board that cannot hold the higher speed: a watchdog notices if the board stumbles and drops it one step on the next boot, with no re-flashing needed. You can find out whether that happened:

```
int mhz = japi_get_cpu_clock_mhz();    // what it is actually running
if (japi_clock_was_reverted())          // the asked-for tier was dropped
    ... ;
```

So if you opt into 390 MHz and the screen shows odd glitches or the board reboots, that board simply cannot hold the top tier; let it settle at 324 and carry on.

One SD-card trap

The SD card is set up on purpose with its "card-detect" line disabled. Do not turn it on. The card driver would install its own background interrupt for it, and that interrupt disturbs the scanline timing the video depends on. The default is already correct; this matters only if you go digging in the storage setup.

Start with japi_init()

As every example has shown, japi_init() comes first, before anything else in main -- it is what brings all of the above to life.

With these few habits in mind you can build something much larger without surprises. The last chapter is a short map of the source code, for when you want to see how any of this actually works.

15. Where to look in the source

Japi Base is meant to be read. The whole machine is a handful of files, small enough to get through in an afternoon, with nothing important hidden behind a driver you cannot open. If you like to learn from working code, here is the map.

Start with the header

japi_base.h is the whole public surface in one place: every function you have used in Part II, plus the named constants -- the colours, the key codes, the musical notes, the file modes. Read it once and you have seen everything a program can call. If you only open one file, open this one.

The engine: japi_base.c

This is the machine itself. It is laid out top to bottom in sections, each with a banner comment you can search for:

HARDWARE PIN CONFIGURATION	which Pico pin is which
CPU CLOCK STATE	the three-tier clock and its watchdog
KEYBOARD ENGINE	PS/2 decoding and the key queue
AUDIO ENGINE	the four-channel synth
VGA / BITMAP / SOUND PUBLIC API	the calls you met in Part II
CORE 1 ENTRY / JAPI_INIT	how the machine boots the engine
LittleFS + UNIFIED FILE I/O API	the A: and C: drives

(The VGA scanline render and the double buffer sit in the middle, the timing-critical core that the rest is built around.) So to see how, say, the keyboard turns a key press into the code japi_get_char hands you, search for "KEYBOARD ENGINE" and read down. The same trick finds the video, the sound or the storage.

The vendored libraries: third_party_libs.c

This file is large, but it is not really ours: it is three outside libraries gathered into one place -- FatFs (the SD-card filesystem), the SD-over-SPI driver and littlefs (the store behind the built-in media, driven by Japi Base's own flash glue). You almost never need to open it. The one part worth knowing is at the very bottom: a small configuration block that wires the SD-card pins named in japi_base.h (PIN_SD_MISO and friends) into the driver. That is how a pin number at the top of the project reaches the hardware at run time -- change a pin in the header and this block carries it through.

The rest

A few smaller files round out the project:

japi_base.pio	the PIO programs that generate the VGA signal and read PS/2 -- the timing-critical heart
font_8x12.h	the 8 x 12 character shapes
japi_kbd_defaults.h	the built-in QWERTY_US layout table
demo.c	the bundled demo -- the biggest single example of the API in the whole project
CMakeLists.txt	the build, including which .c file holds main

That is the entire machine. For a quick reference rather than a tour, the appendices list every pin Japi Base uses and every function in `japi_base.h`. From here the best next step is to change something small, rebuild and watch what happens -- which is what Japi Base is for.

Building on Japi Base

Japi Base exists to be built on. The whole point of a small, open platform is that you fork it and turn it into the base for your own idea -- a synth, a data logger, a little game, any project that wants a screen, a keyboard and storage with nowhere to put a full operating system. If you make something with it, I would genuinely love to hear about it.

And if you hit a bug or see a way to make Japi Base better, please say so. Bug reports and suggestions are welcome as issues on the project's GitHub page (github.com/JanFromBelgium/japi-base); improvements are welcome as pull requests -- you do not need to be an expert to send one. Japi Base itself is written in collaboration with an AI assistant, so we are all learning here. A platform grows faster and stays more useful with other people in it -- so please, come and build.

APPENDICES

A. Quick connection lookup table

Every connection Japi Base makes, in one place. "Pin" is the Pico 2's physical pin number; the series resistors and capacitors are explained in Chapter 2.

VGA -- weighted-resistor DAC, two bits per colour into 75 ohm

GP0	pin 1	VSYNC	-> VGA pin 14	(100R)
GP1	pin 2	HSYNC	-> VGA pin 13	(100R)
GP2	pin 4	Blue, low bit	-> VGA pin 3	(1k)
GP3	pin 5	Blue, high bit	-> VGA pin 3	(470R)
GP4	pin 6	Green, low bit	-> VGA pin 2	(1k)
GP5	pin 7	Green, high bit	-> VGA pin 2	(470R)
GP6	pin 9	Red, low bit	-> VGA pin 1	(1k)
GP7	pin 10	Red, high bit	-> VGA pin 1	(470R)
GND			-> VGA pins 5, 6, 7, 8, 10 and the connector shell	

Audio -- PWM + RC filter per channel (1k + 10uF series, 3.3nF shunt)

GP8	pin 11	Left	-> jack tip
GP9	pin 12	Right	-> jack ring
GND			-> jack sleeve

PS/2 keyboard -- through a bidirectional level shifter

GP14	pin 19	CLOCK	<-> shifter <-> PS/2 pin 5
GP15	pin 20	DATA	<-> shifter <-> PS/2 pin 1

+5 V from VBUS (pin 40) feeds the keyboard and the shifter HV side;
3.3 V (pin 36) and GND feed the shifter LV side.

Micro-SD -- SPI1, 3.3 V native, no shifter

GP10	pin 14	SCK	-> SD CLK
GP11	pin 15	MOSI	-> SD DI
GP12	pin 16	MISO	-> SD D0
GP13	pin 17	CS	-> SD CS

3.3 V (pin 36) -> SD VCC (with a 100nF decoupling cap); GND -> SD GND

Power

3.3 V	pin 36	logic supply (VGA levels, SD, shifter LV side)
VBUS	pin 40	5 V for the keyboard (and the shifter HV side)
GND	several	common ground

Everything not listed -- Core 0, the PI01 and PI02 blocks, the spare DMA channels -- is free for your own programs (see Chapter 14).

This page is intentionally left blank.

B. japi_base.h API reference

Every public function, grouped by area. The full lists of constants -- the colours (VGA_*), key codes (JAPI_KEY_*), notes (NOTE_*), modifier bits (JAPI_MOD_*), waveforms (JAPI_WAVE_*) and file modes -- are in japi_base.h itself.

Start-up

```
void japi_init(void);
    Bring up clock, VGA, keyboard and SD, and start the engine on
    Core 1. Call once, before anything else.
```

CPU clock

```
bool japi_set_cpu_clock(int mhz);
    Persist a speed (260, 324 or 390) and reboot into it. Always safe.
int japi_get_cpu_clock_mhz(void);
    The speed the board is actually running at.
bool japi_clock_was_reverted(void);
    True if the last requested tier was unstable and stepped down.
int japi_clock_reverted_from(void);
    The tier that failed, in MHz, or 0 if none.
```

Screen -- text

```
void vga_clear(uint8_t fg, uint8_t bg);
    Fill every cell with a blank on background bg.
void vga_set_char(int row, int col, uint8_t code, uint8_t fg, uint8_t bg);
    Set one cell (row 0-63, col 0-126) to a character code and colours.
void vga_print(int row, int col, const char *str, uint8_t fg, uint8_t bg);
    Write a string from (row, col), left to right, clipped at the edge.
void vga_update(void);
    Show everything drawn since the last call (waits for the vblank).
void vga_redefine_char(uint8_t code, const uint8_t bitmap[12]);
    Replace a glyph with a 12-byte bitmap (top bit of each byte = left).
```

Screen -- bitmap

```
bool japi_bitmap_open(int col, int row, int w_chars, int h_chars,
                      int scale, bool double_buffered);
    Open a pixel canvas over a block of cells (scale 1 or 2;
    double_buffered for tear-free animation). False if it will not fit
    in free memory or the scale-2 even-width rule is broken.
void japi_bitmap_close(void);
    Release the canvas; the cells return to text.
void japi_bitmap_clear(uint8_t colour);
    Fill the whole canvas with one colour.
void japi_bitmap_pixel(int x, int y, uint8_t colour);
    Set one pixel (x from the left, y from the top).
uint8_t *japi_bitmap_buffer(void);
    The raw canvas bytes (one per pixel) for drawing directly.
int japi_bitmap_width(void);
int japi_bitmap_height(void);
    The canvas size, in logical pixels.
```

Keyboard

```
bool japi_has_char(void);
    True if a key is waiting to be read.
uint16_t japi_get_char(void);
    Take the next key: an ASCII value, or a JAPI_KEY_* code.
bool japi_keydown(uint16_t code);
    True while that key is physically held now (shift-independent, queue-
    independent); pass the base char or a JAPI_KEY_*/JAPI_KEY_MOD_* code.
uint8_t japi_kbd_modifier_state(void);
    A bitmask of the modifiers held right now (JAPI_MOD_*).
```

Sound

```
void japi_play(uint8_t note, uint16_t ms);
    Play a note on channel 0 for ms milliseconds (non-blocking).
void japi_play_ch(int ch, uint8_t note, uint16_t ms);
    The same on a chosen channel (0-3); NOTE_REST stops the channel.
```

```

void japi_sound_wave(int ch, uint8_t type);
    Set the waveform (JAPI_WAVE_SINE / _SQUARE / _SAW / _TRIANGLE).
void japi_sound_freq(int ch, uint16_t hz);
    Set the pitch directly in hertz.
void japi_sound_volume(int ch, uint8_t vol);
    Set the volume, 0 to 255.
void japi_sound_pan(int ch, uint8_t pan);
    Set the stereo position (0 left, 128 centre, 255 right).
void japi_sound_envelope(int ch, uint16_t attack_ms,
                        uint16_t decay_ms, uint8_t sustain,
                        uint16_t release_ms);
    Shape the note over time (ADSR; sustain level 0 to 255).
void japi_sound_note_on(int ch);
void japi_sound_note_off(int ch);
    Start a held note / release it (it then fades over the release).
void japi_sound_off(void);
    Silence all four channels at once.
int japi_audio_stream(const int16_t *buf, int nframes);
    Push mono 16-bit PCM (~48 kHz); returns the frames accepted.
int japi_audio_stream_stereo(const int16_t *buf, int nframes);
    Push interleaved L,R 16-bit PCM; returns the frames accepted.
int japi_audio_stream_space(void);
    Free frames in the stream buffer, so the app can pace itself.
void japi_audio_stream_stop(void);
    Stop streaming; the four-channel synth resumes.

```

Files (A: = SD card, C: = built-in media)

These work through a file handle -- you create a variable of type `japi_file_t` and pass its address to each call.

```

bool japi_fopen(japi_file_t *f, const char *path, uint8_t mode);
    Open a file. mode = JAPI_READ, JAPI_WRITE (replace), JAPI_APPEND, or
    JAPI_READ | JAPI_WRITE for random access (keeps existing data).
int japi_fread(japi_file_t *f, void *buf, int size);
    Read up to size bytes; returns the count, which is 0 at end of file.
int japi_fwrite(japi_file_t *f, const void *buf, int size);
    Write size bytes; returns the count written.
bool japi_fseek(japi_file_t *f, int pos);
    Move the read/write position to byte offset pos (0 = start).
void japi_fclose(japi_file_t *f);
    Close the file (this also flushes a file you wrote).
bool japi_exists(const char *path);
    True if the file exists.
bool japi_remove(const char *path);
    Delete a file.
bool japi_mkdir(const char *path);
    Create a folder.
bool japi_rmdir(const char *path);
    Remove an empty folder.
bool japi_rename(const char *from, const char *to);
    Rename or move a file or folder within one drive (not across drives).
int japi_fsize(japi_file_t *f);
    The size of an open file, in bytes.

```

Directories

A listing uses a `japi_dir_t` handle, the same way as the file handle above.

```

bool japi_opendir(japi_dir_t *d, const char *path);
    Open a directory for listing.
bool japi_readdir(japi_dir_t *d, char *name_out, int name_max);
    Get the next entry name; false when done ( "." and ".." are skipped).
void japi_closedir(japi_dir_t *d);
    Close the directory.

```

C. Licence and credits

Japi Base is released under the BSD 3-Clause License (the full text is in the LICENSE file). You are free to use, study, change and share it, including in closed-source and commercial work; the only conditions are that you keep the copyright notice and do not use the project's name to endorse a version of your own.

The libraries gathered into `third_party_libs.c` keep their own original notices, in the licence block at the top of that file:

FatFs (ChaN)	BSD 1-clause style
SD-over-SPI driver (Carl Kugler III)	Apache 2.0
littlefs	BSD-3-Clause

The built-in media (drive C:) runs on Japi Base's own flash glue on top of littlefs, so every part of the combined binary now sits under a permissive BSD- or Apache-style licence. (Earlier releases pulled in the GPL-3 pico-lfs adapter, which forced the whole binary to GPL-3; it has since been replaced and Japi Base relicensed from GPL-3 to BSD-3-Clause.)

The demo's full-screen image is The Starry Night (Vincent van Gogh, 1889), which is in the public domain, via Wikimedia Commons and the Google Art Project.

Finally: Japi Base is made by Jan, in collaboration with Claude, an AI assistant. Thanks for reading -- now go and build something.